Document Number	**PPD-CMPE-131 Sec xx**
Version	**1.0**
Status	**Draft**
Created By	**[Taha Khan]**

OurSJSU

Project Plan Document

Reviewers

Department	**Name/Title**
Engineering	Evan Ha
System Architecture	Christopher Nguyen
Product Management	Taha Khan
Marketing	Min Khant Htun
Quality Assurance (QA)	Rico Li
Engineering	Kyle Ringor

Team Member Contribution Record
Project Team: After Six
Deliverable: Project Plan Document
Each team member must participate in the development of this deliverable. As a team, review and agree on each member's contribution level and complete the table below before submission.

Team Member	Contribution Level	Brief Description of Contribution
Evan Ha	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 9
Christopher Nguyen	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 8
Rico Li	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 3 and 6
Taha Khan	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 1, 2, 4
Kyle Ringor	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 5
Minh Khant Htun	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 7

Contribution Levels
Full Contribution (100%) – Fully participated and contributed to the deliverable; receives 100% of the team score.
Partial Contribution (50%) – Partially participated or contributed to the deliverable; receives 50% of the team score.
No Contribution (0%) – Did not participate or contribute to the deliverable; receives 0% of the team score.

Team Confirmation:
By submitting this document, the team confirms that the contribution levels reported above have been reviewed and agreed upon by the team members.

Table of Contents

1 Introduction
- 1.1 Purpose of the Plan
- 1.2 Scope of the Project
- 1.3 Assumptions and Constraints

2 Project Organization
- 2.1 Roles and Responsibilities
- 2.2 Organizational Structure

3 Risk Management
- 3.1 Risk Identification
- 3.2 Risk Analysis
- 3.3 Contingency Plan

4 Project Schedule
- 4.1 Milestones
- 4.2 Timeline

5 Resource Allocation
- 5.1 Budget
- 5.2 Human Resources
- 5.3 Tools and Technologies

6 Monitoring and Reporting
- 6.1 Progress Monitoring
- 6.2 Reporting

7 Quality Management MKH
- 7.1 Quality Standards
- 7.2 Testing Plan

8 Project Closure
- 8.1 Deliverables
- 8.2 Review
- 8.3 Post-Project Activities

9 Appendices

List of Tables

Table 1.1:

Table 1.2:

1 Introduction

1.1 Purpose of the Plan

- The purpose of this plan is to create a project plan that details the scope, assumptions, Project Organization, Risk Management, Project Schedule, Resource Allocation, Monitoring and Reporting, Quality Management, and Project Closure of OurSJSU.
- This document will serve as a reference for the Product Manager, the Project Team, and Stakeholders to know the overall plan for this project.

1.2 Scope of the Project

- The boundaries of the project are similar to what has been covered before in the Product Requirements and Team Specification Documents. The scopes of this term shall be: Creating a web application, connecting necessary external APIs for new features, furnishing a user-friendly UI, and building on older previous features, the details of which have been covered in the PRD.
- The expected outcome of this project is a fully functioning OurSJSU Web Application with all the modules listed in the Team Specification Document and Product Requirements Document.

1.3 Assumptions and Constraints

- Assumptions: We operate on the assumption that everyone contributes fully in their role, that the requirements will not be subject to vast revision, that the necessary external component APIs are available, and that the Stakeholders do not defund the project.
- Constraints: Restrictions include: Deadlines, Stakeholder Funding, External API functionality and availability.

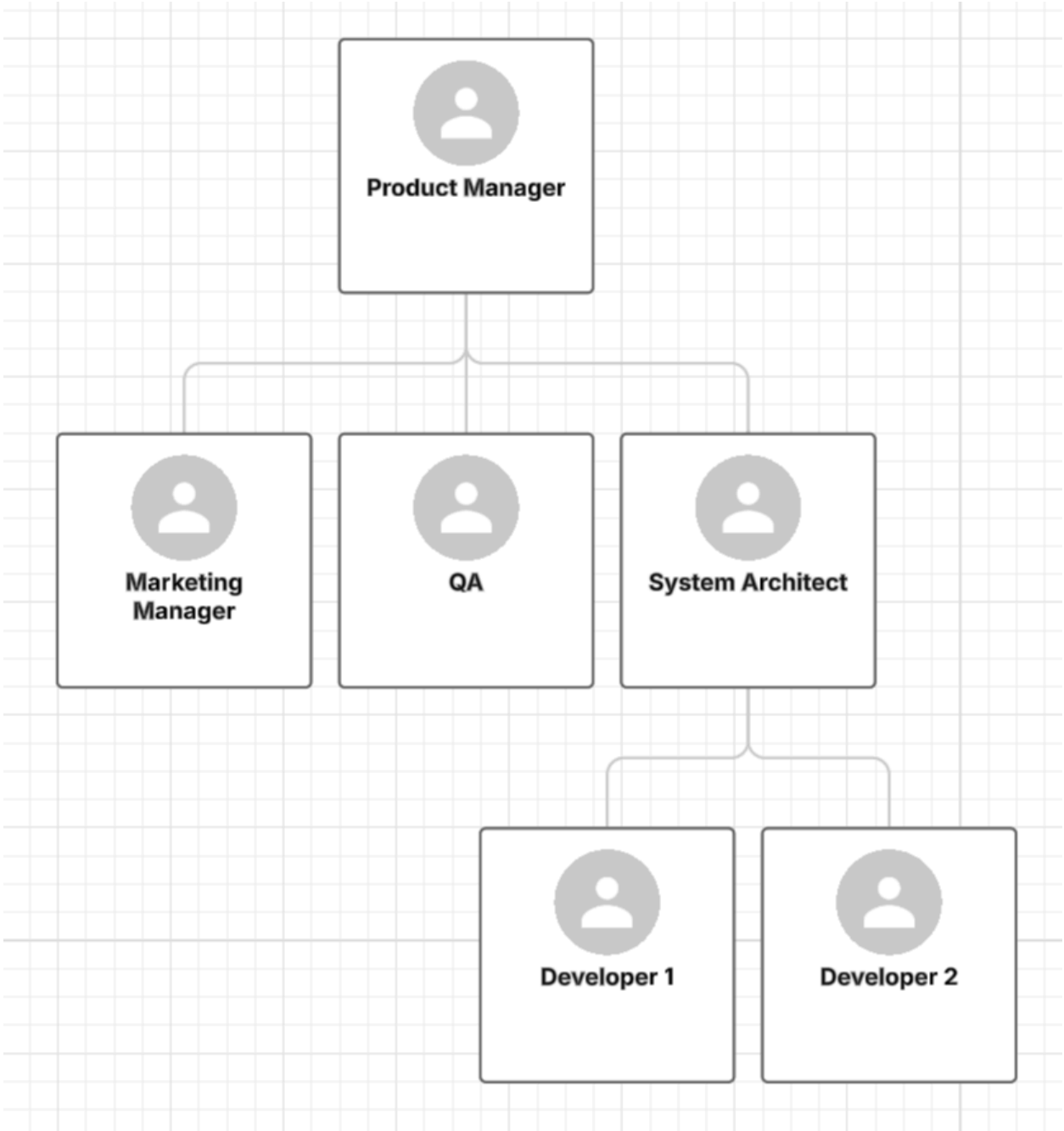
2 Project Organization

2.1 Roles and Responsibilities

Role	Name/Department	Responsibilities

Project Manager	Taha Khan/Product Management	Coordinate Overall Project Timeline, Develop and Maintain Sprint Plan, Track Budget, Respond to certain Stakeholder Concerns
Product Marketing Manager	Minh Khant Htun/ Marketing	Collect User Information, advertise the project to generate interest, review requirements and connect external feedback to internal work.
System Architect	Christopher Nguyen/ System Architecture	Determine and update the technical structure of the system. Determine the interactions between components of the system. Develop diagrams to illustrate the architecture of the system.
Software Developer	Kyle Ringor, Evan Ha / Engineering	Develop the front-end UI/UX and the back-end of the system with the requirements listed in all technical documentation,
QA Engineer	Rico Li/ Quality Assurance	Test the code and make sure the code aligns with the typical quality a user expects.

2.2 Organizational Structure



3 Risk Management

3.1 Risk Identification

- **Technical Risks**:

Web Scraping Failure & Data Schema Changes:

Unannounced structural or layout changes to official SJSU Course Catalog or Class Schedule HTML pages could break the web scraper and prevent course/seat updates.

Third-Party API Latency or Outages:

Third-party notification delivery services (Email, SMS, Discord, Slack) could experience downtime or rate limiting, impacting the 99% delivery success rate requirement.

High Concurrency Traffic Spikes:

Heavy student traffic during peak registration periods could lead to backend database bottlenecking or system slowdowns.

- **Business Risks**:

  Scope Creep:

  Attempting to implement features outside the defined scope could lead to project delays.
- **Project Risks**:

  Schedule & Resource Bottlenecks:

  Conflicts in team member availability or delays in delivering core modules could delay overall milestones.

3.2 Risk Analysis

Risk	Likelihood	Impact	Mitigation Strategy
SJSU Web Structure Changes (Scraper Failure)	Medium	High	Implement exponential backoff retry mechanisms and system error logging. Serve the last cached catalog data in case of parsing failures so core services remain functional.
Notification Dispatch Failure (Email/SMS/Discord)	Low	High	Maintain multi-channel redundancy. If a primary channel fails, log the error and attempt delivery via secondary enabled channels
System Overload during Peak Registration	Medium	High	Utilize a layered MVC client-server architecture with optimized SQL queries. Containerize the system using Docker for future cloud-hosted autoscaling during high-demand windows

Incomplete Functional and Integration Testing	Low	Medium	Execute comprehensive unit and integration test suites using a dedicated SQLite test environment, ensuring 100% test coverage for all HIGH-priority requirements
Project Schedule Delays	Medium	Medium	Assign clear Person-in-Charge (PIC) roles for each deliverable, hold regular progress meetings, and track development against strict milestone deadlines

3.3 Contingency Plan

Data Ingestion Interruption:
If SJSU schedule pages undergo major layout updates, the system automatically degrades gracefully to "Cached Data Mode," displaying the most recent valid course listings while firing a high-priority alert in the audit logs for developer attention.

Authentication Outage:
If the SJSU SSO endpoint is temporarily unreachable, the system will prompt an error message while maintaining active session tokens for already logged-in users.

Notification API Rate-Limiting:
If SMS provider limits are hit, notifications will automatically reroute to Webhook-based channels (Discord/Slack) or Email SMTP.

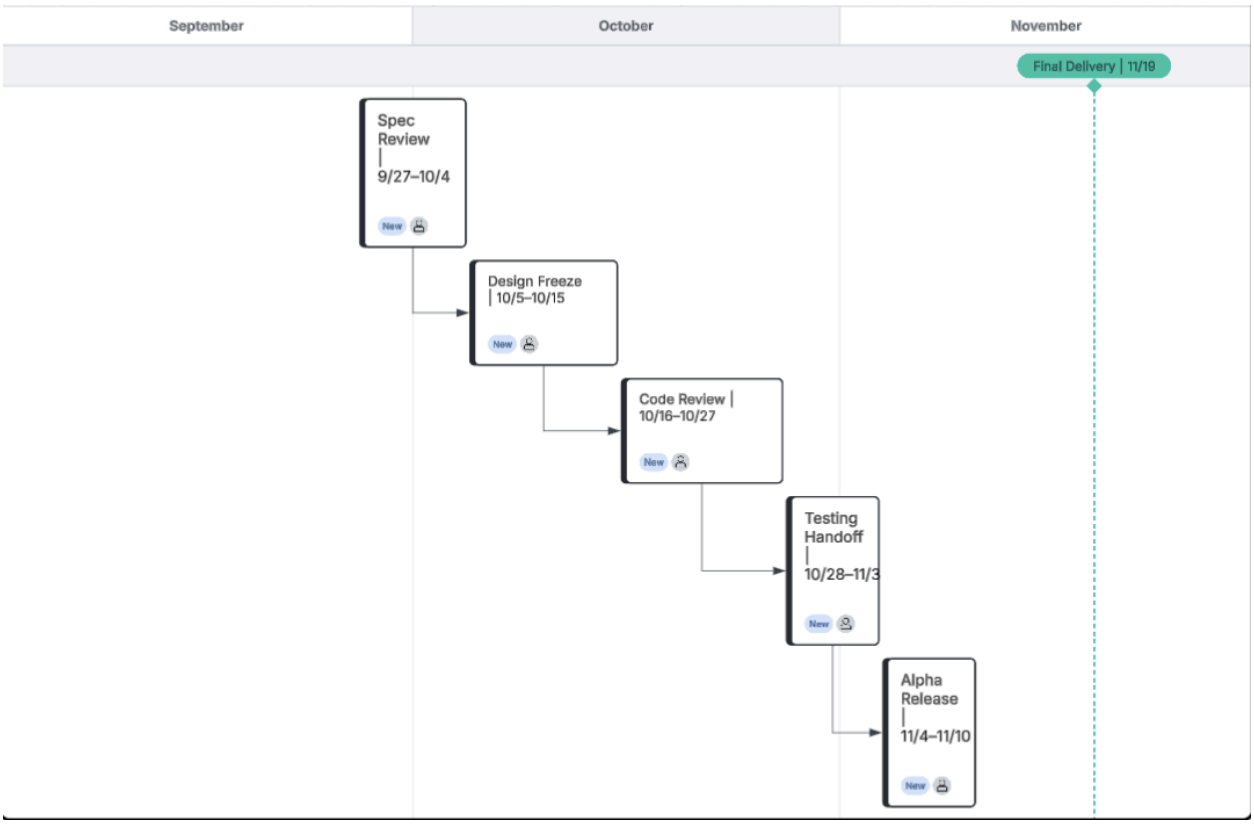
4 Project Schedule

4.1 Milestones

Milestone	Description	Due Date
Specification Review	Stakeholder approval of system requirements.	October 4, 2026
System Design Freeze	Final approval of system design.	October 15, 2026
Code Review	Code review and unit test report.	October 27, 2026

Testing hand-off	Engineering release testing.	November 3, 2026
Alpha Release	First working version of the software.	November 10, 2026
Final Delivery	Deployment of the completed system.	November 19, 2026

4.2 Timeline



5 Resource Allocation

5.1 Budget

Category	Description	Cost
Development	Labor for two developers, a system architect, a project manager, and product marketing.	$18,400
QA/Testing	QA labor, test data, and a test environment.	$4,000
Deployment	Hosting, email and SMS alerts, and a domain.	$1,700

5.2 Human Resources

- **Project Manager, Taha Khan:** Manages the schedule, budget, task assignments, and project reviews. 60 hours.
- **Product Marketing, Min Khant Htun:** Collects student feedback, reviews the user interface, and prepares user-facing materials. 40 hours.
- **System Architect, Christopher Nguyen:** Designs the system structure, database, security controls, and external integrations. 80 hours.
- **Software Developer, Kyle Ringor:** Develops the student interface, course search, and degree planning features. 160 hours.
- **Software Developer, Evan Ha:** Develops backend services for course data, watchlists, and notifications. 160 hours.
- **QA Engineer, Rico Li:** Creates test cases, records defects, and verifies fixes. 120 hours.

5.3 Tools and Technologies

- **Development tools:** Visual Studio Code will be used to write code. Python will support backend functions and course data collection. SQLite will store data. Git and GitHub will be used for version control and code reviews.
- **Testing tools:** Python unit tests and a separate SQLite test database will be used to test application logic. The team will test the interface in Chrome. Sample records will be used to test application functionality.
- **Deployment tools:** Docker will package the application for deployment on a hosted server.

6 Monitoring and Reporting

6.1 Progress Monitoring

- **Tracking Mechanisms**:
  - **Weekly Team Meetings and Standups**:
    - Regular team discussions to review project status, allocate tasks, and address blockers.
  - **Version Control & GitHub Tracking**:
    - Monitor feature progress, pull requests, and commit history in GitHub.
- **Performance & Progress Metrics**:
  - **Tasks Completed and Planned**:
    - Tracking feature completion against the core modules specified in the PRD and SFDS.
  - **Defect and Test Coverage Tracking**:
    - Evaluating unit test results, bug resolution rates, and requirement validation using the test scenarios in SFDS Section 11.

6.2 Reporting

- **Report Types**:
  - **Weekly Status Reports**:
    Summarizing completed deliverables, upcoming tasks, budget tracking, and identified risks.
  - **Milestone Review Reports**:
    Presenting project status at formal checkpoints:
    Specification Review, System Design Freeze, Code Review, Testing Hand-off, Alpha Release, Final Deliver.

- **Reporting responsibilities**
  Project Manager (Taha Khan):
  Tracks project schedules, updates budgets, delegates action items, and leads milestone reviews.
  System Architect (Christopher Nguyen)
  and Software Developers (Kyle Ringor, Evan Ha):
  Provide technical progress updates, architectural documentation, and code review reports.
  QA Engineer (Rico Li): Prepares test execution reports, bug logs, and validation results.
  Product Marketing Manager (Min Khant Htun):
  Gathers user requirements, updates UI/UX feedback, and reviews documentation alignment.

7 Quality Management

7.1 Quality Standards

- **Test Coverage:** Maintaining 100% test coverage for all high functional requirements mentioned in the PRD and SFDS.
- **System Reliability & Uptime:** To achieve a 99% successful delivery rate for seat notification alerts across all available channels (Email, Discord , Slack, SMS).
- **Fault Tolerance & Degradation:** To guarantee zero system downtime during scraper failures by falling back to cached catalog data while logging errors with exponential backoff.
- **Performance & Scalability:** Optimize SQL query execution and API endpoints to ensure page response times remain under 2 seconds during peak registration periods.
- **Code & Style Standards:** Strict adherence to project coding guidelines:
  - **Naming Conventions**: PascalCase for classes, and snake_case for variables, functions and methods.
  - **Version Control**: All code changes must be submitted via separate Git with descriptive messages and undergo code reviews before merging into main.
  - **Documentation & Clean Code**: Consistent indentation, docstrings and comprehensive inline comments across Python backend scripts.

7.2 Testing Plan

- **Unit Testing:**
    - **Authentication & Sessions:** Validate session token generation, role assignment, expiration and invalid password error..
    - **Prerequisite Evaluator:** Verify logic evaluation (AND/OR tree traversal) for completed, in-progress and missing course rules.
    - **Roadmap Engine:** Test degree progress calculations, course completion mapping and recommendation sorting.
    - **Data Model & API Validation:** Ensure malformed JSON payloads, foreign key constraints, and invalid section IDs are rejected with clear HTTP error responses.

- **Integration Testing:**
    - **Search & Comparison:** Validate multi-filter queries (subject, instructor, open_seats) and cross section matrix generation.
    - **Watchlist & Seat Monitoring:** Test adding/removing watched sections, duplicate prevention and availability change event triggers.
    - **Notification Dispatch:** Verify multi channel alert delivery (Email, Discord, Slack) upon seat availability changes, confirming provider failure retry loops do not block other active channels.
- **System Testing:**
    - Complete user workflows in Google Chrome.
    - Verify system stability under database error conditions, scraper failure fallbacks and API limiting scenarios.

- **Testing Timeline:**

Phases	Tasks	Checkpoints
Phase 1 :Test Setup	Develop unit test cases for core algorithms (Prerequisites, Roadmap, Authentication)	Weeks 4-5
Phase 2 : Component & Unit Testing	Execute unit tests for API endpoints, SQLite schema validation, and data models	Weeks 6-7
Phase 3 : Integration & System Testing	Test Scraper-Monitor-Watchlist pipeline, notification dispatchers, and UI flows	Weeks 8-9

Phase 4 : Verification & Release	Log defects, execute bug-fix verifications, and deliver final QA Test Report	Weeks 10-11

8 Project Closure

8.1 Deliverables

- **Deployed Software:** Stakeholders will receive the access to the website and ability to use all of its features
- **Source Code and Repository:** Source code will be posted in a GitHub repository, allowing other developers to improve OurSJSU, or make their own version
- **User Guides:** Along with the application, stakeholders will receive multiple guides for the proper way to use the application.

8.2 Review

- Project success will be defined by the reception by stakeholders and the functionality of features during actual academic terms.
    - At the minimum, during the time period where SJSU students sign up for classes, features related to class enrollment need to work (class section changes, waitlist tracker, notification sender) for the project to be deemed successful
- Quantitative Success Criteria:
    - **Notification Delivery:** >95% notification delivery success rate across all platforms
    - **Performance and Scalability:** Page response time under 2 seconds during peak registration traffic spikes
    - **Fault Tolerance:** Zero downtime during SJSU web catalog structure changes, verified by cache data
    - **Prerequisites Confirmation:** Prerequisite checker will remain accurate if any changes to the SJSU Computer Engineering Catalog occur

8.3 Post-Project Activities

- **Operational Handover & Maintenance:** The project will be maintained by a smaller team of maintenance developers to update course information for future academic terms
    - Transition system administration, database maintenance scripts, and scraper update procedures (for handling SJSU catalog changes) to the dedicated maintenance team.
- **Financial & Resource Reconciliation**: Construct a final financial audit comparing actual expenses to expected expenses and budget
- **Future Roadmap Expansion:** Document Phase 2 scaling goals, such as expanding degree roadmap support beyond Computer Engineering majors to other SJSU engineering departments.

9 Appendices

- **Detailed Schedule**:
  - Project Milestones:

Milestone	Due Date
Specification review	October 4, 2026
System Design Freeze	October 15, 2026
Code Review	October 27, 2026
Testing Hand-off	November 3, 2026
Alpha Release	November 10, 2026
Final Delivery	November 19, 2026

  - Testing Schedule:

Phase	Tasks	Timeline
Test Setup	Create unit tests for authentication, prerequisites, and roadmap functions.	Weeks 4-5
Component and Unit Testing	Test API endpoints, SQLite schema, and data models. Complete the code review.	Weeks 6-7
Integration and System Testing	Test the scraper, monitor, watchlist, notifications, and UI workflows. Complete testing hand-off and alpha release.	Weeks 8-9
Verification and Release	Record defects, verify fixes, prepare the QA report, and complete final delivery.	Weeks 10-11

- **Glossary**:

Term	Definition
API	Application Programming Interface; a method that allows software systems to communicate.
Backend	The part of the application that processes requests, applies system logic, and accesses data.
Client-Server Architecture	A design where clients send requests to a server that processes and returns information.
Concurrency	The ability to handle multiple users or system operations during the same period.
Defect	A problem in the software that causes incorrect or unexpected behavior.
Deployment	The process of releasing an application so users can access it.
Exponential Backoff	A retry method that increases the waiting time after each failure.
Fault Tolerance	The ability of a system to continue operating when a component fails.
Foreign Key	A database field that connects related records in different tables.
HTTP	Hypertext Transfer Protocol; used to exchange information between web components.
Integration Testing	Testing that verifies different system components work together correctly.
JSON	JavaScript Object Notation; a structured format used to exchange data.
Milestone	A scheduled checkpoint representing an important stage of the project.
MVC	Model-View-Controller; separated system data, the user interface, and application logic.
Notification Dispatch	The process of sending alerts through supported communication channels.
PRD	Product Requirements Document; defines the project's features and requirements.
Pull Request	A request to review code changes before merging them into the

	main codebase.
Rate Limiting	A restriction on how many requests can be sent during a set period.
Scalability	The ability of a system to support an increased number of users or requests.
Scope Creep	The addition of unplanned work or features beyond the approved project scope.
SFDS	Software functional Design Specification; describes the detailed software design.
SMTP	Simple Mail Transfer Protocol; used to send email notifications
SQL	Structured Query Language; used to retrieve and manage database information.
SSO	Single Sign-On; allows users to authenticate through an existing account provider.
Stakeholder	A person or group affected by or interested in the project.
Test Coverage	A measurement of how much required functionality is tested.
UI/UX	User Interface and User Experience; describes the system's appearance and usability.
Unit Testing	Testing individual functions, classes, or components separately.
Version Control	A system for tracking and managing changes to source code.
Webhook	A method for automatically sending information between applications after an event.

- **Review meeting minutes**: time, attendees, reviewer/review comments, action items/PICs.
    - Time: September 22, 2026, 2:15-2:45 PM
    - Attendees: Rico Li, Christopher Nguyen, Min Khant Htun, Evan Ha, Kyle Ringor
    - Reviewer/Review Comments:
        - Reviewed Sections 3, 4, and 5 of the PPD
        - Discussed Risk Management to better understand framework
        - Confirmed that the team must stay on track with timeline
        - Divided PPD work among team members

- AI/PICs: Finish PPD, PICs: Rico Li, Christopher Nguyen, Min Khant Htun, Evan Ha, Kyle Ringor, Taha Khan, Due Date: 9/27/2026